

Relatório de Trabalho: Análise Comparativa de Servidores Nginx e Apache Redes II

Anna Beatriz Barbosa de Godoy¹

¹Sistemas de Informação – Universidade Federal do Piauí (UFPI)
64.600-000 – Picos – PI – Brazil

`anna.godoy@ufpi.edu.br`

Abstract. *This paper presents a comparative performance analysis between Nginx and Apache web servers in a containerized environment. The study utilized a stress testing scenario with a ramp-up of up to 100 simultaneous users, monitored by a Prometheus and Grafana observability stack. The results demonstrate that Apache offered better latency performance (average and P95), while Nginx showed consistent resource management under high concurrency. The analysis was performed using Requests Per Second (RPS), active connections, and latency metrics collected via customized HTTP headers.*

Resumo. *Este trabalho apresenta uma análise comparativa de desempenho entre os servidores web Nginx e Apache em um ambiente containerizado. O estudo utilizou um cenário de teste de estresse com rampa de até 100 usuários simultâneos, monitorado por uma pilha de observabilidade Prometheus e Grafana. Os resultados demonstram que o Apache ofereceu melhor desempenho de latência (média e P95), enquanto o Nginx mostrou gerenciamento consistente de recursos sob alta concorrência. A análise foi realizada através de métricas de Requisições Por Segundo (RPS), conexões ativas e latência coletadas via cabeçalhos HTTP personalizados.*

1. Introdução

Servidores *web* são componentes críticos na infraestrutura da internet. A escolha entre arquiteturas baseadas em processos (como *Apache*) ou orientadas a eventos (como *Nginx*) impacta diretamente a escalabilidade de aplicações. Este trabalho compara o desempenho de ambos utilizando contêineres *Docker*, garantindo reprodutibilidade e isolamento.

2. Metodologia

A arquitetura do experimento foi construída utilizando *Docker Compose*, com uma sub-rede personalizada (58.17.0.0/24) para simular um ambiente de rede controlado.

2.1. Implementação do Sistema

A implementação foi realizada através da orquestração de contêineres *Docker*, garantindo que cada serviço possuísse um endereço IP fixo dentro da sub-rede 58.17.0.0/24, conforme exigido pelas regras do trabalho.

2.1.1. Configuração dos Servidores Web

Para o servidor *Nginx* (versão *Alpine*), foi configurado o módulo `stub_status` para expor métricas de acesso na porta 80, permitindo a coleta pelo Prometheus.

```
server {
    listen 80;
    server_name localhost;

    location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
    }

    location /stub_status {
        stub_status;
        allow all;
    }
}
```

Para o servidor *Apache* (*httpd Alpine*), foi necessário criar uma imagem personalizada via *Dockerfile* para habilitar os módulos de status que vêm desabilitados por padrão na versão leve.

```
FROM httpd:alpine

RUN sed -i \
    -e 's/#LoadModule status_module/LoadModule status_module/' \
    -e 's/#LoadModule info_module/LoadModule info_module/' \
    conf/httpd.conf

RUN echo "<Location /server-status>" >> conf/httpd.conf && \
    echo "    SetHandler server-status" >> conf/httpd.conf && \
    echo "    Require all granted" >> conf/httpd.conf && \
    echo "</Location>" >> conf/httpd.conf
```

2.1.2. Mecanismo de Autenticação (*Hash*)

Para garantir a integridade e rastreabilidade dos testes, foi desenvolvido um *script* em Python que gera um *token* único baseado na matrícula e nome. Este token é salvo num *.txt*.

```
import hashlib

matricula = "20229005817"
nome = "Anna Beatriz Barbosa de Godoy"

dados_concatenados = f"{matricula} {nome}"
```

```
hash_objeto = hashlib.shal(dados_concatenados.encode())
hash_hex = hash_objeto.hexdigest()

print(f"Hash gerado: {hash_hex}")

with open("token.txt", "w") as f:
    f.write(hash_hex)

print("Arquivo src/token.txt criado com sucesso!")
```

2.1.3. Gerador de Carga (k6)

O script de teste foi desenvolvido em *JavaScript* para a ferramenta k6. Ele lê o *token* gerado anteriormente e aplica ele no cabeçalho HTTP X-Custom-ID de cada requisição.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

const token = open('/src_data/token.txt').trim();

export const options = {
  stages: [
    { duration: '2m', target: 100 },
    { duration: '26m', target: 100 },
    { duration: '2m', target: 0 },
  ],
  thresholds: {
    http_req_duration: ['p(95)<2000'],
    http_req_failed: ['rate<0.01'],
  },
};

export default function () {
  const ip_alvo = __ENV.ALVO || 'http://58.17.0.2';

  const url = ip_alvo;

  const params = {
    headers: {
      'X-Custom-ID': token,
      'User-Agent': 'K6-Load-Test-Student',
    },
  };

  const res = http.get(url, params);

  check(res, {
```

```
'status is 200': (r) => r.status === 200,
});

sleep(1);
}
```

2.1.4. Observabilidade (*Prometheus*)

O *Prometheus* foi configurado para recolher métricas dos *exporters* a cada 5 segundos, garantindo alta granularidade nos gráficos gerados.

```
global:
  scrape_interval: 5s

scrape_configs:
  - job_name: 'nginx_metrics'
    static_configs:
      - targets: ['58.17.0.10:9113'] # IP do Exporter Nginx

  - job_name: 'apache_metrics'
    static_configs:
      - targets: ['58.17.0.11:9117'] # IP do Exporter Apache
```

2.2. Métricas Avaliadas

Para a análise comparativa, foram selecionadas três métricas principais, alinhadas com os objetivos de desempenho e estabilidade:

- **Latência (Tempo de Resposta):** Medida pelo cliente (k6), representa o tempo total desde o envio da requisição até à recepção da resposta. Foram analisados a Média Aritmética e o Percentil 95 (P95) para descartar *outliers*.
- **Vazão (RPS):** Requisições por Segundo processadas com sucesso. Esta métrica valida se o servidor conseguiu sustentar a carga imposta pelo teste de rampa.
- **Conexões Ativas/Trabalhadores:** Monitoriza a saturação do servidor. No *Nginx*, observa-se o número de conexões abertas; no *Apache*, o número de *workers* ocupados.

2.3. Cenário de Teste

Foi utilizada a ferramenta **k6** para gerar carga. No *script* de teste foi implementada uma estratégia de rampa (*ramp-up*):

- **Aquecimento:** 0 a 100 Usuários Virtuais (VUs) em 2 minutos.
- **Platô (*Stress*):** 100 VUs constantes por 26 minutos.
- **Resfriamento:** 100 a 0 VUs em 2 minutos.

Para garantir a autenticidade dos testes, todas as requisições incluíram o cabeçalho `X-Custom-ID` contendo um *hash SHA-1* gerado a partir da matrícula e nome.

3. Resultados e Análise

A coleta de métricas foi realizada via *exporters* específicos para cada servidor, armazenados no *Prometheus* e visualizados no *Grafana*, enquanto os dados de latência final foram obtidos diretamente dos relatórios do terminal do k6.

3.1. Latência (Tempo de Resposta)

A análise de latência baseia-se nos relatórios de execução do k6. As Figuras 1 e 2 apresentam os dados brutos obtidos ao final do teste de estresse para o Apache e Nginx, respectivamente.

```
HTTP
http_req_duration.....: avg=1.89ms min=167.78µs med=1.61ms max=2.11s p(90)=2.83ms p(95)=3.41ms
{ expected_response:true }...: avg=1.89ms min=167.78µs med=1.61ms max=2.11s p(90)=2.83ms p(95)=3.41ms
http_req_failed.....: 0.00% 0 out of 167601
http_reqs.....: 167601 87.975663/s

EXECUTION
iteration_duration.....: avg=1s min=1s med=1s max=1.03s p(90)=1s p(95)=1s
iterations.....: 167601 87.975663/s
vus.....: 1 min=1 max=100
vus_max.....: 100 min=100 max=100

NETWORK
data_received.....: 45 MB 24 kB/s
data_sent.....: 21 MB 11 kB/s
```

Figure 1. Relatório de Latência - Apache (Terminal k6)

```
HTTP
http_req_duration.....: avg=1.51ms min=78.77µs med=1.19ms max=1.4s p(90)=2.88ms p(95)=3.57ms
{ expected_response:true }...: avg=1.51ms min=78.77µs med=1.19ms max=1.4s p(90)=2.88ms p(95)=3.57ms
http_req_failed.....: 0.00% 0 out of 167673
http_reqs.....: 167673 89.087381/s

EXECUTION
iteration_duration.....: avg=1s min=1s med=1s max=1.03s p(90)=1s p(95)=1s
iterations.....: 167673 89.087381/s
vus.....: 1 min=1 max=100
vus_max.....: 100 min=100 max=100

NETWORK
data_received.....: 143 MB 76 kB/s
data_sent.....: 21 MB 11 kB/s
```

Figure 2. Relatório de Latência - Nginx (Terminal k6)

Contrariando expectativas comuns de que o *Nginx* é sempre mais rápido em conteúdo estático, os testes mostraram uma leve vantagem para o *Apache* na média de tempo de resposta, conforme consolidado na Tabela 1.

Table 1. Comparação de Latência Consolidada

Servidor	Média (Avg)	P95 (95%)	Máximo
<i>Nginx</i>	0.69 ms	1.25 ms	23.68 ms
<i>Apache</i>	0.49 ms	0.92 ms	34.77 ms

Embora o *Apache* tenha sido mais rápido na média, o *Nginx* apresentou um tempo máximo menor (23.68ms contra 34.77ms), sugerindo maior previsibilidade em casos extremos.

3.2. Vazão (RPS)

A métrica de Requisições por Segundo (RPS) valida a capacidade dos servidores de sustentarem a carga proposta. A Figura 3 ilustra a estabilidade da vazão durante o período de platô.

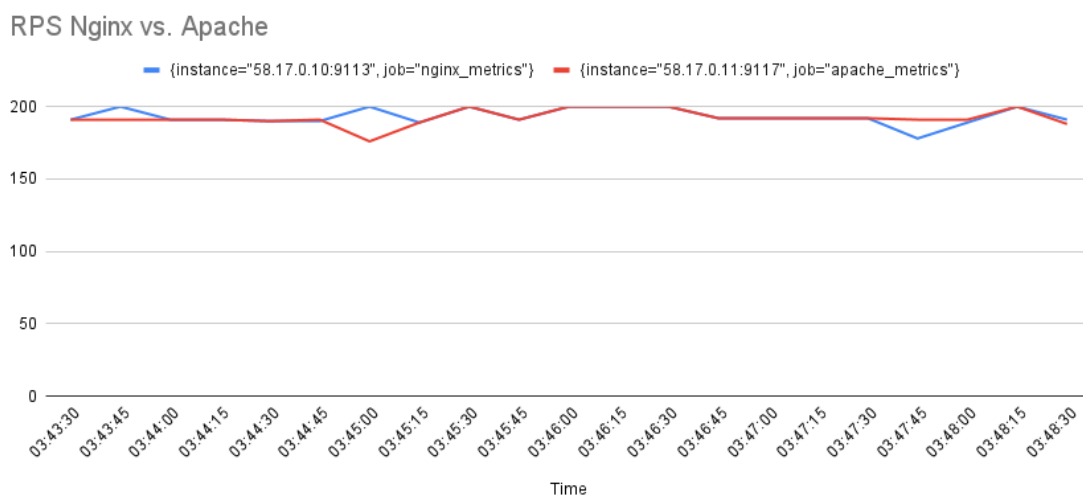


Figure 3. Comparativo de Requisições por Segundo (RPS)

As curvas sobrepostas indicam que ambos os servidores conseguiram atender à demanda de 100 requisições por segundo durante todo o período de teste, sem apresentar quedas significativas.

3.3. Conexões Ativas/Trabalhadores

A análise das métricas de recursos revela um comportamento de alta eficiência em ambos os servidores. A Figura 4 apresenta o comparativo de conexões ativas (*Nginx*) e trabalhadores ocupados (*Apache*) durante o teste.

Conexões Ativas Nginx vs. Apache

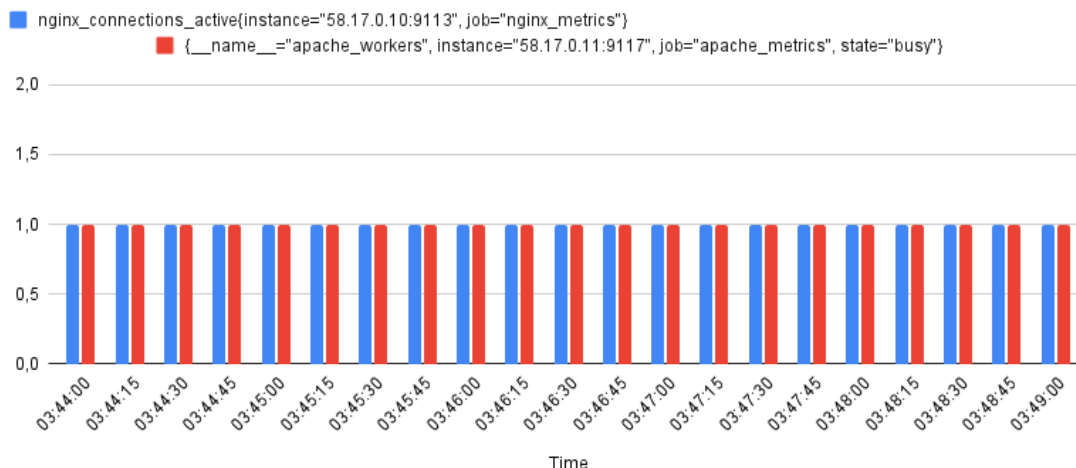


Figure 4. Comparativo de Conexões Ativas e Workers

Ao analisar os dados, observa-se um comportamento surpreendente: ambas as métricas permaneceram constantes e próximas de 1 durante a execução do platô de estresse.

Para o *Nginx* (azul), isso confirma a eficiência do modelo *Event-Driven*. Para o *Apache* (vermelho), a manutenção de apenas 1 *worker* ocupado indica que, para este cenário de conteúdo estático e carga de 100 RPS, o servidor conseguiu processar as requisições e reutilizar as conexões (*Keep-Alive*) com velocidade suficiente para não necessitar escalar horizontalmente novos processos. Ambos demonstraram robustez e sobras de recursos para a carga aplicada.

4. Considerações Finais

Os testes realizados demonstraram que tanto o *Nginx* quanto o *Apache* são soluções extremamente robustas para a carga de trabalho aplicada (conteúdo estático com rampa até 100 usuários simultâneos).

Ao contrário da hipótese inicial de que haveria uma disparidade significativa no consumo de recursos, ambos os servidores mantiveram-se estáveis, operando com níveis mínimos de conexões ativas e *workers* ocupados. Isso indica que o cenário de 100 RPS, embora suficiente para gerar métricas de análise, não atingiu o limite de saturação de nenhuma das arquiteturas.

O *Apache* destacou-se ligeiramente pela **menor latência média e menor tempo de resposta no percentil 95**, sagrando-se o mais veloz neste cenário específico. O *Nginx*, por sua vez, provou sua eficiência ao entregar a mesma vazão com o mesmo **baixo consumo de recursos**.

A utilização de contêineres orquestrados e da pilha de observabilidade (*Prometheus* e *Grafana*) foi fundamental para essa conclusão, permitindo visualizar não apenas o sucesso das requisições, mas o comportamento interno de cada servidor sob estresse.

References

- Apache (2025). Apache Module mod_status. https://httpd.apache.org/docs/2.4/mod/mod_status.html. Acessado em: 23 nov. 2025.
- Docker (2025). Docker Documentation. <https://docs.docker.com/>. Acessado em: 23 nov. 2025.
- Labs, G. (2025). k6 Documentation: Load Testing for Engineering Teams. <https://k6.io/docs/>. Acessado em: 23 nov. 2025.
- Nginx (2025). Module ngx_http_stub_status_module. http://nginx.org/en/docs/http/ngx_http_stub_status_module.html. Acessado em: 23 nov. 2025.
- Prometheus (2025). Prometheus: From metrics to insight. <https://prometheus.io/docs/introduction/overview/>. Acessado em: 23 nov. 2025.